COMPONENT BASED SOFTWARE ENGINEERING

Component Composition, Integration and COTS

Component composition

- Component composition is a design pattern where you build complex user interfaces by combining smaller, independent, and reusable components like Lego bricks. Instead of writing a single monolithic component bloated with dozens of props and conditional checks, you distribute responsibilities across cooperating parent and child component
- Component composition is the process of combining two or more software components to form a complete software system or application.
- In component-based architecture, each component performs a specific function and interacts with other components through defined interfaces.
- The aim is to assemble components so that they work together to satisfy the requirements of a software system.

Methods of component composition

1. Sequential composition: Components are connected in a sequence, where the output of one component becomes the input of another. Example: A component validates data before another component processes it.

2. Hierarchical composition: Components are organized into levels, where a higher-level component coordinates lower-level components. Example: An order management component coordinates payment and delivery components.

3. Additive composition: Components are combined to add new functionality to an existing system. Example: Adding a notification component to an existing mobile application.

4. Parallel composition: Components perform different tasks concurrently or independently.

Example: After an order is placed, separate components process inventory updates and customer notifications.

Note: These are common ways to describe component composition; terminology and classifications may differ across course materials.

Steps involved in component composition

- Identify the system requirements.
- Identify or select suitable software components.
- Examine component interfaces and dependencies.
- Connect compatible components using appropriate connectors.
- Integrate the components into a complete system.
- Test the integrated system to ensure that the components work correctly together.

Advantages of component composition

- Reusability: Existing components can be reused in different applications.
- Reduced development time: Developers do not need to build every function from scratch.
- Maintainability: Individual components can be modified independently when interfaces remain compatible.
- Flexibility: Components can be replaced or added to extend functionality.
- Improved productivity: Development work can be divided among different teams.

Component Integration

- **component integration** is the process of assembling independent, reusable software units into a single cohesive system.
- Because these components are often built by different teams or third-party vendors, integration focuses heavily on ensuring they can successfully communicate, share data, and interoperate without conflicting

Objectives of Component Integration

- To combine individual software components into a complete system.
- To enable communication between different components.
- To ensure that components work together as expected.
- To identify and resolve compatibility problems.
- To promote software reuse and reduce development time.
- To ensure that the integrated system meets user requirements.

Cont.:

Integration covers several distinct concerns:

- **Interfaces** — Matching APIs, data formats, protocols, and contracts between components. Mismatches here (type differences, calling conventions, unit mismatches) are the most common source of integration bugs.
- **Wiring/glue code** — The adapters, connectors, and orchestration logic that let components talk to each other without modifying the components themselves.
- **Lifecycle & state** — Managing initialization order, shared state, startup/shutdown sequencing, and dependency resolution.
- **Non-functional concerns** — Threading/concurrency models, error propagation, transaction boundaries, logging, security, and performance across component boundaries.

Methods of Component Integration

METHOD	EXPLANATION
Top-down integration	Begins with higher-level components and progressively integrates lower-level components.
Bottom-up integration	Begins with lower-level components and combines them into higher-level components.
Big-bang integration	Combines most or all components at once before testing the integrated system.
Incremental integration	Integrates and tests components gradually, one at a time or in small groups.

The 3 Core Stages of Integration

Before components can be seamlessly woven together, developers follow a sequence of distinct actions to prepare them:

- **Component Qualification:** Assessing whether a pre-built component meets the system's functional and non-functional requirements.
- **Component Adaptation:** Modifying or wrapping the component if it does not fully align with the target architecture or interface requirements.
- **Component Composition:** The actual assembly phase where components are systematically linked together to form the integrated application

Common Integration Challenges

While integration speeds up development, it also introduces several critical bottlenecks:

- **Architectural Mismatch:** Occurs when a reusable component makes implicit, hidden assumptions about its environment (e.g., its control model or memory management) that conflict with the overall system design.
- **Interface Incompatibility:** Differences in communication protocols, parameter counts, or data formats that require extensive glue code to fix.
- **Versioning Chaos:** Upgrades or security patches in a single third-party component can create a ripple effect, breaking backward compatibility with dependent parts of the system

Interface Matching

Interface matching is the process of ensuring that the interfaces of different software components are compatible so that they can communicate and exchange information correctly.

- Key activities:
- Checking whether the output of one component matches the input requirements of another.
- Ensuring that components use compatible method names, parameters, and return types.
- Verifying that communication protocols are supported by the connected components.
- Adapting incompatible interfaces using adapters or wrappers.

Example: A payment component expects an amount in Kenyan shillings, while the ordering component supplies the amount in a different format. Interface matching ensures that the components exchange the required information correctly.

Dependency management

- Dependency management in component-based engineering (CBE/CBA) is the discipline of ensuring that separately developed components can be assembled and deployed together without conflicts, missing pieces, or version clashes. It sits at the heart of integration because components are rarely self-contained — they depend on other components, libraries, runtime services, and configuration data that must all be present and compatible at deployment time.
- dependency management means identifying, declaring, controlling, and resolving what each component needs in order to work, so components can be assembled, replaced, and upgraded without breaking the system.
- Components are built independently, so at integration time you discover what each one *actually* needs: other components, libraries, runtime services, configuration, or specific versions. Unmanaged dependencies lead to failed assembly, unpredictable behavior, and components that can’t be reused or replaced.

Types of Dependencies

- **Functional dependencies:** Component A calls services (required interfaces) that component B provides.
- **Structural/build dependencies:** Libraries, frameworks, or packages needed to compile or package the component.
- **Runtime/environment dependencies:** Operating system, container, middleware, database, or configuration the component expects.
- **Data dependencies:** Shared schemas, databases, or file formats.
- **Temporal/protocol dependencies:** Required call order, startup order, or timing between components.
- **Transitive dependencies:** Dependencies of your dependencies, which you inherit without choosing.

Main Problems

PROBLEM	DESCRIPTION
Version conflicts	A needs library v1, B needs v2 (“dependency hell”)
Circular dependencies	A depends on B and B depends on A, so neither can be deployed or tested independently
Hidden dependencies	Undocumented reliance on global state, environment variables, or internal behavior
Tight coupling	Component depends on a concrete implementation instead of an interface
Dependency bloat	Pulling in large libraries for small features, widening the attack surface
Missing dependencies	Required component or service absent in the target environment

Common Challenges in Dependency Management

- Missing components or libraries.
- Incompatible component versions.
- Circular dependencies, where components depend on each other in a cycle.
- Changes to one component affecting other components.
- External services becoming unavailable.
- Security risks arising from outdated or vulnerable dependencies.

Data compatibility

- **Data compatibility** refers to ensuring that data exchanged between components is understood consistently on both sides—despite differences in data models, formats, or timing semantics.
- Since components are treated as black boxes communicating through required and provided interfaces, each interface is equipped with a data model defining the types occurring in interface operations.
- When these data models differ, integration fails unless adaptation is applied.

Cont.:

- Data incompatibilities arise at several levels:
  - **Syntactic mismatches**: Different field names, structures, or data representations (e.g., XML schemas that define the same data differently).
  - **Streaming behavior**: One component passes data one record at a time (streamed), while another expects the entire dataset at once (non-stream).
  - **Timing/resolution differences**: Components update data at different intervals, or data paths have varying transit times, leading to inconsistent data sets

Common Problems

- **Format mismatch:** Different serialization or file formats (XML, JSON, CSV, binary).
- **Schema mismatch:** Different field names, structures, or cardinality.
- **Type mismatch:** Precision loss, overflow, truncation (e.g., floating point for currency).
- **Encoding errors:** Garbled text, especially with non-ASCII characters.
- **Semantic mismatch:** Data parses fine but means something different to each side. This is the hardest to detect.
- **Schema evolution:** A component changes its data structure and breaks existing consumers.
- **Null and missing value handling:** One side uses null, another uses empty string or a default value.
- **Data quality and integrity:** Different validation rules, constraints, or reference data (e.g., country codes).

Activities Involved in Data Compatibility

- 1. Data type matching: Ensure that components use compatible data types when exchanging information.
- Example: One component sends a student's age as an integer, while another expects a numeric value of the same compatible type.
- 2. Data format standardization: Ensure that components use consistent formats when exchanging data.
- Example: One component uses JSON to exchange student records, and another component is configured to receive JSON.
- 3. Data structure matching: Ensure that components agree on the organization and arrangement of data fields.
- Example: A student component provides student_id, name, and course, while the receiving component expects these fields under the same agreed structure.

Activities Involved in Data Compatibility

- 4. Data conversion: Convert data from one representation to another when components use different formats or types.
- Example: Converting a date from 10/10/2026 to 2026-10-10 before sending it to another component.
- 5. Data validation: Check that exchanged data meets the receiving component's requirements.
- Example: Ensuring that a student's admission number is provided and follows the required format before saving the record.
- 6. Data consistency: Ensure that components use consistent values and meanings for shared information.
- Example: Two components should use the same agreed currency, such as Kenyan shillings (KES), when processing a student's fee payment.

Integration mechanisms

- **Integration mechanisms** are the technical means, techniques, tools, and communication patterns used to connect independently developed software components so that they can communicate, exchange data, and coordinate their behaviour to form a unified software system.
- They define *how* components interact during system assembly after their interfaces have been matched and dependencies resolved. Components may communicate within the same application or across different computers and networks.
- Since a key principle of CBA is to promote loose coupling and independent component development and deployment, selecting an appropriate integration mechanism influences the system's performance, scalability, reliability, resilience to failures, and ease of maintenance and component replacement.

Main Mechanisms

1. Direct API / Procedure Call

- An API provides a defined set of operations through which one software component can request services or exchange information with another component. One component invokes another's interface within the same process or runtime.
- Defines how components interact, Specifies the operations and data that can be exchanged and Reduces the need for components to know each other's internal implementation.
- **Examples:** Library calls, method invocation, .NET assemblies.

2. Remote Invocation (RPC / RMI)

- RPC allows one component to invoke a procedure or operation provided by another component, potentially running on a different computer, as though it were a local call.
- Supports communication between distributed components, Hides some of the details of network communication, Requires suitable communication infrastructure and Calls cross process or network boundaries but look like local calls.
- **Examples:** gRPC, Java RMI, CORBA, DCOM.

Cont.: Main Mechanisms

3. Service-Based Integration (Web Services)

- Components are exposed as services over standard protocols with published contracts. Web services enable software components to communicate over a network using standardized web technologies and protocols.
- Allow components built using different programming languages to communicate, Commonly use HTTP for communication and May exchange data using JSON or XML.
- **Examples:** REST, SOAP, GraphQL; microservices.

4. Message-Based / Event-Driven Integration

- Components communicate asynchronously through a message broker or event bus, using queues or publish/subscribe. Message queues allow components to exchange messages asynchronously. The sending component places a message in a queue, and the receiving component processes it when available.
- Reduces the need for components to be active simultaneously, Supports communication between loosely coupled components and Helps manage temporary delays in processing.
- **Examples:** Kafka, RabbitMQ, ActiveMQ, cloud queues.

COTS (Commercial Off-The-Shelf)

- **Commercial Off-The-Shelf (COTS)** components are pre-existing, ready-made software or hardware products developed by vendors and made available to the general market for purchase, lease, or licensing. In Component-Based Architecture (CBA) and Component-Based Engineering (CBE), these components are integrated into larger systems to provide existing functionality instead of developing everything from scratch.
- COTS products are generally designed to meet the needs of a broad range of users rather than the specific requirements of a single organization. They are typically used as supplied or configured to suit the system's needs, without modifying their internal source code, which is often unavailable to users and controlled by the supplier.

Cont.: COTS (Commercial Off-The-Shelf)

- Examples of COTS components include database management systems such as Oracle Database and Microsoft SQL Server, payment gateways, Customer Relationship Management (CRM) systems, Enterprise Resource Planning (ERP) packages, reporting tools, and user interface component libraries.
- Integrating COTS components can reduce development effort and accelerate software delivery. However, developers must consider potential challenges, including interface incompatibility, dependency conflicts, data compatibility, security requirements, licensing restrictions, and dependence on the vendor for updates and maintenance.

COTS in Component Integration

- During integration, developers must ensure that a COTS component is compatible with the system's interfaces, dependencies, data formats, and security requirements.
- Adapters or wrappers may be necessary to resolve incompatibilities.

Integration Challenges

COTS integration is fundamentally different from custom development. As IEEE Software observed, "dealing with COTS is a high-risk activity for a variety of reasons" .

- **Architecture mismatch** is the initial manifestation that integration is anything but simple. COTS components make assumptions about architectural issues; when these assumptions conflict with your system or with other components, the simplicity of "buy not build" disappears in a cloud of integration scaffolding .
- **Glue code** becomes necessary to repair mismatched assumptions, but this code is "often ad hoc and brittle." As a consequence, COTS-based systems can be difficult to comprehend, less evolvable than intended, and less reliable than the constituent products .
- **Evolution costs** are substantial. Research with industry practitioners found that "a large amount of work is required to integrate new releases of COTS software into their systems." For systems with multiple COTS components on independent release schedules, "the cost of integrating each new release of each component becomes prohibitive" .
- **Late discovery** is common: "Many of the problems encountered with integrating COTS components cannot be determined before integration begins. The problems are of such a nature that they only appear well into the integration process

Integration Strategies

- **Treat components as black boxes** and rely on external techniques for resolving mismatches . Since you cannot modify the component, you must adapt around it.
- **Use wrappers and glue** to allow for interface complications in future releases, "even if not strictly needed now" . Think of them as gaskets and washers between mechanical components.
- **Select for integration mechanisms** such as public APIs and scripting languages of appropriate quality when evaluating products .
- **Manage implicit dependencies:** If one COTS component depends on the presence of another, the dependency can break and the system can fail. Select components to minimize implicit dependencies and be aware they exist .
- **Plan releases strategically:** Preplan releases at approximately six-to-nine month intervals, with multiple releases simultaneously in progress, to manage the influx of vendor updates

Characteristics

- **Pre-built and pre-tested:** Already developed and used by other customers.
- **Black-box nature:** You get the interface and documentation, usually not the source code.
- **Vendor-controlled:** The vendor decides features, release schedule, and end-of-life.
- **Generic:** Designed for many customers, so it may not fit your needs perfectly.
- **Licensed:** Use is governed by commercial terms, not ownership.

Advantages of COTS

- Reduced development time: Ready-made functionality reduces the need to build software from scratch.
- Lower initial development cost: Reusing an existing product may cost less than developing an equivalent component.
- Proven functionality: Established products may have undergone extensive testing and use.
- Vendor support: Vendors may provide updates, documentation, and technical assistance.
- Faster deployment: Products can often be installed and configured quickly.

Disadvantages of COTS

- Limited customization: The product may not fully meet specific user requirements.
- Integration difficulties: Its interfaces or data formats may be incompatible with existing components.
- Vendor dependence: Continued use may depend on the vendor's pricing, support, and product availability.
- Licensing costs: Purchase, subscription, or renewal fees may be required.
- Security and compatibility risks: Updates or changes may introduce vulnerabilities or compatibility problems.

End

!!!!!!!